OuicheFS Extended Blocks

Design and Status Report

Jonathan Remarque · Hakan Eyilmez · Amer Redzovic
Linux Kernel Programming (LKP), RWTH Aachen University · July 2026

Overview

This project extends OuicheFS with a more flexible way of organizing file data. Instead of limiting a file to a fixed list of individual blocks, consecutive blocks are grouped into extents. This supports larger and sparse files, encourages contiguous allocation, and provides the basis for reservations, filesystem statistics, and defragmentation.

1. Design choices: algorithms and data structures

Extents and file layout

Each file has one 4 KiB index block containing up to 512 extent entries. An extent describes a consecutive range of blocks, so a large continuous file needs only a small number of entries. The same structure records gaps in sparse files without allocating data blocks for those gaps. Keeping the index to one block preserves OuicheFS's simple disk format while allowing files to grow beyond the former 1024-block limit.

Contiguous allocation and reservations

When data is written, the allocator first looks for a consecutive free range and joins adjacent allocations into the same extent. If the complete range is unavailable, a shorter range can still be used. When a file has no reservation left, the requested allocation is the blocks needed by the current write plus the configured reservation size. The whole returned run first enters the reservation pool, and the current write then consumes blocks from its front. The configured size is therefore a target for future writes rather than a strict cap on this temporary pool, and a shorter allocation can still be used for partial progress. This reduces interleaving between writers and helps sequential files remain contiguous. Unused reservations are released, and a cleanup pass can reclaim them when the filesystem runs out of free space.

Defragmentation

Defragmentation tries to replace each fragmented data region with one contiguous run. A region is changed only after the new blocks have been allocated and copied successfully; otherwise, its original layout is kept. Processing then continues with the next region. This all-or-nothing decision for each region protects file contents, while the best-effort approach can still improve the remaining parts of a file or partition.

2. Implementation status

Implemented and functional

- **Read and write support.** File operations handle offsets, EOF, partial blocks, append mode, writes across block boundaries, truncation, and writes beyond the former EOF.
- **Extent format and ioctl.** The new 512-entry extent index is used by mkfs, read, write, truncate, and inode deletion. OUICHEFS_IOC_GET_EXTENTS reports physical runs and reservations.
- **Contiguous allocation.** The allocator coalesces adjacent blocks into extents and accepts shorter runs when the complete request cannot be satisfied.
- **Write-time reservation and GC.** Per-inode reservation state reduces interleaving between writers; unused blocks are released on close/eviction, and the GC reclaims held reservations under ENOSPC.
- **Per-partition statistics.** The required free, committed, reserved, file, extent, average-size, maximum-size, fragmentation, reservation-size, and GC-run values are exposed in /sys/ouichefs/<device>/.
- **Sparse files.** Hole extents consume no disk blocks, read as zeros, can be appended or merged, and are split into prefix/data/suffix entries when a write lands inside an existing hole.
- **Defragmentation.** OUICHEFS_IOC_DEFRAG_FILE relocates data into contiguous runs while preserving holes and content. Automatic best-effort defragmentation is triggered above the configured fragmentation threshold.

Implemented with known limitations

- **Loaded-inode scope.** Automatic partition defragmentation and maximum-file-size recomputation iterate sb->s_inodes. Consequently, files whose inodes are not currently cached may be omitted. A complete fix would scan the on-disk inode store and acquire each regular inode safely.

Not implemented

No mandatory roadmap feature was omitted. The optional dedicated sysfs file for the automatic-defragmentation threshold was not added; the threshold remains available as the writable fragmentation_threshold module parameter. The optional bonus advanced allocator was also not implemented. Allocation therefore still scans the bitmap linearly for a suitable run instead of maintaining a buddy-style structure with size-classed free ranges.

3. Tests and reproduction

Tests ran in the provided QEMU virtual machine with an Arch Linux userspace, 1 GiB of RAM, and Linux 6.6.137. A 50 MiB test device was formatted as OuicheFS and mounted at /mnt/ouichefs. The following commands rebuild the module and formatter, load the module, format the disposable test disk, and execute the complete suite:

```
make && make -C mkfs
insmod ./ouichefs.ko
./mkfs/mkfs.ouichefs /dev/sdc
mount -t ouichefs /dev/sdc /mnt/ouichefs
MNT=/mnt/ouichefs ./extent.sh
```

The 33 shell tests cover single- and multi-block reads and writes, append and truncate semantics, reads beyond EOF, large sequential files, extent ioctls, contiguous allocation, reservation release, concurrent-writer GC, all sysfs statistics, sparse reads, writes at the beginning/middle/end of holes, extent-table exhaustion, and manual defragmentation. The tests compare data with cmp, inspect extent output in dmesg, and repeatedly verify free + committed + reserved = total blocks.

Test-development note. The team first established the test structure and hand-wrote shared helpers for printing, assertions, statistics, and cleanup. AI was then given specific scenarios and asked to draft tests within that framework. Each generated test was reviewed for logical consistency and verified by running it against the implementation.

The performance benchmark compared vanilla OuicheFS, which uses the Linux kernel's generic file operations, with the final extent implementation. Each version ran one warm-up followed by nine measured 3 MiB operations on the same QEMU device. Caches were dropped before cold measurements, and medians were used to limit the effect of individual outliers.

4. Experimental results

- **Overall.** All 33 test cases passed on the final implementation.
- **Capacity and allocation.** A sequential file larger than 4 MiB was written and read back correctly. Reservation testing kept the interleaved large writer in one contiguous extent.
- **Sparse files and GC.** Hole reads returned zeros, all three hole-splitting positions preserved surrounding data, unused reservations were released, and a write on a full partition succeeded after GC reclaimed reserved blocks.
- **Defragmentation.** In the controlled two-file experiment, fragmentation decreased from 400 (4.00 extents per file) to 100 (1.00 extent per file), while byte-for-byte content comparisons passed.

Performance benchmark

Median throughput in MiB/s (higher is better):

Implementation	Buffered write	Durable write	Cold read	Warm read
Kernel-default OuicheFS	49.54	49.58	57.67	191.94
Final implementation	5.51	5.66	15.75	187.34

Here, kernel-default refers to vanilla OuicheFS using Linux's generic file operations. Compared with this baseline, the final implementation delivered 88.9% lower buffered-write throughput, 88.6% lower durable-write throughput, and 72.7% lower cold-read throughput. Warm-read throughput remained close, with a loss of 2.4%. The added control and filesystem features therefore come mainly at a cost in writing and uncached reading, while cached reads remain comparable.

5. Conclusion

The project successfully replaces OuicheFS's flat block index with an extent-based design and adds contiguous allocation, reservations, sparse-file support, statistics, and defragmentation. All functional tests passed. In the controlled fragmentation test, defragmentation reduced the layout from 4.00 to 1.00 extents per file without changing the file contents. The benchmark shows that this additional functionality comes with a clear performance cost: compared with vanilla OuicheFS, writes and cold reads are substantially slower, while warm-read throughput remains close. The main remaining limitation is that some partition-wide operations only scan loaded inodes. Scanning the on-disk inode table would make these operations complete for files that are not currently cached.